

Q1 Student Management System [BL3 – Apply]

10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

Your Answer

```
START
CLASS student
PRIVATE:
    name
    roll_no
    mark1,mark2,mark3
    average
METHOD student details(name,roll_no, mark)
    this.name<-name
    this.roll_no<-roll_no
    this.mark1<-mark1
    this.mark2<-mark2
    this.mark3<-mark3
END METHOD
METHOD calculate average()
    average<-(mark1+mark2+mark3)/3
END METHOD
METHOD display result()
    IF average >=50 then
        print "pass"
    ELSE
        print "fail"
    END IF
END METHOD
END CLASS
MAIN
CREATE OBJECT s OF student
INPUT name,roll_no
INPUT mark1,mark2,mark3
CALL s.students details(name,roll_no,mark1,mark2,mark3)
CALL s.calculate average()
CALL s.display result()
STOP
```

 Save Answer

Q2 2. Library Book Management [BL3 – Apply]

10 marks

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.

OOP Concepts: Encapsulation | Getters & Setters | Objects

Your Answer

```
START
create class book
PRIVATE:
    title
    author
    ISBN
    isavailable
methods:
setdetails(title, author, ISBN)
    store title,author,ISBN
    isavailable=TRUE
issuebook()
    IF isavailable=TRUE THEN
        isavailable=FALSE
        print "book issued"
    ELSE
        print "book not available"
    ENF IF
returnbook()
    isavailable=TRUE
    print "book returned"
displaydetails()
    print title
    print author
    print ISBN
    print availability status
END class
MAIN program
create book1,book2,book3
set detailsfor all 3 books
call book1.issuebook()
call book2.issuebook()
call book1.returnbook()
call display details()for book1
call display details()for book2
call display details()for book3
STOP
```

Q3 Employee Salary Calculation [BL3 – Apply]

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empid, name, basicPay, designation). Implement methods to:

Calculate gross salary (basic + HRA + DA)

- Calculate net salary after deductions (PF, tax)

- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

Your Answer

```
START
create employee class
private data:
    empid
    name
    designation
    basicpay
create methods:
    setdetails()
    calculategross()
    calculatenet()
    displaypayslip()
setdetails()
    read empid,name,designation,basicpay
calculategross()
    HRA=basicpay*20/100
    DA =basicpay*10/100
    gross=gross-(PF+tax)
displaypayslip()
    print empid
    print name
    print designation
    print basicpay
    print gross
    print net
MAIN program
    create employee object
    call setdetails()
    call calculatenet()
    call displaypayslip()
STOP
```

 Save Answer

Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]

10 marks

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.

OOP Concepts: Inheritance | Polymorphism | Method Overriding

Your Answer

```
START
create base class shape
method calculatearea()
create class circle extends shape
    input radius
    area=3.14 x radius x radius
    display area
create class rectangle extends shape
    input length,breath
    area=(base x heighth)/2
    display area
create class triangle extends shape
    input base,height
    area=(base x height)/2
    display area
main program
    create circle object
    call calculatearea()
    create rectangle object
    call calculatearea()
    create trianlge object
    call calculatearea()
STOP
```

 Save Answer

Q5 Vehicle Registration System [BL3 – Apply]

10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

OOP Concepts: Inheritance | Method Overriding | Subclass

Your Answer

```
START
create class vehicle
  data:
    regno
    owner
    fueltype
  method displaydetails()
create class twowheeler extends vehicle
  data:
    enginecapacity
  override displaydetails()
    print regno
    print owner
    print fueltype
    print enginecapacity
create class fourwheeler extends vehicle
  data:
    seatingcapacity
    ACstatus
  override displaydetails()
    print regno
    print owner
    print fueltype
    print seatingcapacity
    print ACstatus
MAIN program
  create twowheeler object
  call displaydetails()
  create fourwheeler object()
  call displaydetails()
STOP
```

Save Answer

Q6 Bank Account Hierarchy [BL4 – Analyze]

10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

OOP Concepts: Inheritance | Method Overriding | Polymorphism

Your Answer

```
START
create class bankaccount
  data:
    accno
    holder
    balance
  method withdraw()
  method display()
create class savingaccount extends bankaccount
  data:
    interest
  override withdraw()
    if balance >= amount
      balance = balance - amount
    else
      print "overdraft limit exceeded"
create class currentaccount extends bankaccount
  data:
    overdraftlimit()
  override withdraw()
    if balance + overdraftlimit >= amount
      balance = balance - amount
    else
      print "overdraft limit exceeded"
MAIN program
  create savingaccount object
  call withdraw()
  call display()
  create currentaccount object
  call withdraw()
  call display()
STOP
```

[Save Answer](#)

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

Your Answer

```
START
create class patient
  data:
    patientid
    name
    diagnosis
    prescription
  method generatebill()
create class inpatient extends patient
  data:
    wardnumber
    admissiondate
    dailycharges
  override generatebill()
    bill=dailycharges x numberofdays
    display bill
create class outpatient extends patients
  data:
    appointmentdate
    consultationfee
  override generatebill()
    bill=consultationfee
    display bill
MAIN program
  create inpatient object
  call generatebill()
  create outpatient()
  call generatebill()
STOP
```

Save Answer

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

Your Answer

```
productid
productname
price
method applydiscount()
create class electronics extends product
data:
    warranty
    brand
    voltage
override applydiscount()
    discount=price x 10/100
    finalprice=price - discount
    display finalprice
create class clothing extends product
data:
    size
    fabric
    gender
override applydiscount()
    discount=price x 2-/100
    finalprice=price - discount
    display finalprice
create grocery extends product
data:
    expirydate
    weight
override applydiscount()
    discount = price x5/100
    finalprice=price - discount
    display finalprice
MAIN program
create electronics object
call applydiscount()
create clothing object
call applydiscount()
create grocery object
call applydiscount()
STOP
```

Save Answer

Q9 University Staff Hierarchy [BL4 – Analyze]

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure. Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

Your Answer

```
START
create class staff
  data:
    staffid
    name
    salary
  method calculateallowance()
create class techingstaff extends staff
  data:
    subject
  override calculateallowance()
    allowance=salary x 20/100
    display allowance
create class professor extends teachingstaff
  override calculateallowance()
    allowance=salary x 30/100
    display allowance
create class lecturer extends teachingstaff
  override calculateallowance()
    allowance = salary x15/100
    display allowance
create class nonteachingstaff extends staff
  data:
    department
  override calculateallowance()
    allowance= salary x 10/100
    display allowance
MAIN program
  create proffesor object
  call calculateallowance()
  create lecturer object
  call calculateallowance()
  create nonteachingstaff object
  call calculateallowance()
STOP
```

Q10 Smart Home Device Controller [BL4 – Analyze]

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

Your Answer

```
START
create class smartdevice
  method executecommand()
create class smartlight inherits smartdevice
  override executecommand()
  display "light on"
create class smartthermostat inherits smartdevice
  override executecommand()
  display "thermostat on"
create class smartlock inherits smartdevice
  private PIN
  override executecommand()
  display "door locked"
create objects:
  smartlight
  smartthermostat
  smartlock
call executecommand() for all objects
```

 Save Answer